

From netlist to verified model

Automatic real-number-model and testbench generation
for mixed-signal verification

Veylon Systems

Abstract

A real number model is a fast stand-in for an analog block. Rather than solving transistor physics the way SPICE does, it computes an output number from an input number — so it runs in an ordinary digital simulator, alongside the chip's logic, fast enough for a full regression suite.

The trouble is that somebody has to write it, keep it matching the circuit as the circuit changes, and convince the team it still matches. The first two are slow. The third usually never happens at all.

spice2rnm does all three from one command. Give it a SPICE netlist and you get back a SystemVerilog real number model, a UVM-MS verification environment that exercises it, and the evidence that both agree with the real circuit.

Three things make that evidence worth reading. Every expected value in every generated testbench is a *SPICE measurement of your circuit*, never the model's own equations, so a test cannot pass by agreeing with itself. Every generated environment is proven able to fail: the tool breaks the model on purpose and confirms the test goes red. And how a block gets checked depends on what that block is for — a filter as a frequency response, a duty-cycle corrector by where its edges land, a phase interpolator by how far it moves the clock at each setting.

This paper walks through four circuits, getting harder as it goes. Every number in it was produced by the shipped tool, in about half an hour on a laptop.

Contents

1	The problem	2
2	What you get from one command	2
3	Case study 1 — Two-pole filter	3
4	Case study 2 — Duty-cycle corrector	3
4.1	A model that follows the circuit's dynamics	3
4.2	Verified by function, not by formula	4
5	Case study 3 — Phase interpolator	4
6	Case study 4 — Phase-locked loop	4
6.1	Checked as a system, against the transistors	5
6.2	The rail the deck declares is the rail the model sees	6
6.3	Where the block works is measured, not assumed	6
7	Results at a glance	7
7.1	What the equivalence folders hold	8
8	What the equivalence check compares	9
8.1	What keeps a figure from flattering itself	10
8.2	Blocks, and then the system	10
8.3	One thing a score at an operating point cannot see	11

9	Why a green testbench is worth believing	12
10	Functional specifications as input	13
11	Joining an existing model library	13
12	AI where it helps, physics where it decides	14
13	Scope and requirements	15

1 The problem

A model of an analog block has to be two things at once, and they pull against each other. Fast, or it cannot run in a digital regression suite at all. Accurate, or passing it does not mean anything.

Hand-written models drift away from the circuits they stand for, and the drift is silent. Nothing breaks. The model still compiles, still simulates, still passes its tests. It has simply stopped describing the circuit.

The usual defence is a hand-written testbench, which has the same problem one level up. A person decides what to check and how close is close enough, and those decisions are rarely written down beside the numbers they produced. When the circuit is revised, the model, the testbench and the tolerances all need revisiting by hand — which, in practice, is not what happens.

spice2rnm regenerates all three from the netlist whenever you ask it to. The model, the environment that checks it, and the evidence trail all come out of the same command, so they cannot quietly fall out of step with the circuit or with each other.

2 What you get from one command

```
spice2rnm dcc.cir --input-node clk --output-node vout \
--emit-uvm-ms --spec dcc_spec.md
```

- **A SystemVerilog real-number model**, with the model structure chosen automatically to match the block’s measured behaviour — including blocks whose dynamics move with signal level, where any fixed-response model is wrong across most of its range.

It presents **wreal** ports by default, so it wires straight into an RNM library like the models beside it. That boundary is a module around a computing core, and the core is emitted with plain **real** ports. A **real** port is a *variable*: it depends on no package, no nettype declaration and no simulator extension, so the core compiles standalone anywhere. A flow without **wreal** support instantiates it directly. The boundary can instead be a user-defined nettype from your own library (Section 11), or omitted with **--model-ports real**.

The core is untouched in every case: it is the file the equivalence check ran against. A boundary costs nothing in fidelity either. Driven with identical stimulus, boundary and bare core agree exactly — worst difference 0 over 120 randomised samples across all three case-study models, and again on a library’s own nettype. **wreal** is a simulator extension rather than IEEE 1800, so that boundary is a separate file the core does not depend on.

- **A UVM-MS verification environment** following the Accellera UVM-MS specification, built on IEEE 1800 — not on Verilog-AMS. Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7) with a generated resolution function. Multiple drivers on a net

then combine in the net type rather than in hand-edited testbench code — an offset source, a noise injector, a second bridge. **wreal** is a simulator extension rather than IEEE 1800, so the generated UVM-MS environment drives the core rather than the **wreal** boundary, and needs no AMS licence to run. Bridge and wrapper ports are **interconnect** (§6.6.8), inheriting the testbench’s nettype instead of naming one, and scoreboard golden values are SPICE measurements of your circuit.

- **Equivalence evidence:** circuit and model driven with the identical stimulus, then compared in the domain that matters for the block — waveform error for signal-path blocks, edge placement for clock-path blocks. The aligned traces are archived, so any verdict can be re-derived from its own run directory.

An optional plain-language functional specification supplies what no netlist can: the block’s intended operating range and its acceptance criteria (Section 10).

3 Case study 1 — Two-pole filter

A two-pole RC low-pass filter — the ordinary case, a block that takes a signal in and shapes it on the way out. The generated model is checked four independent ways.

Chirp equivalence	0.0290 against a 0.15 threshold	PASS
PRBS (independent stimulus)	0.0097 normalised	PASS
UVM-MS, DC section	49 checks, 0 failed	
UVM-MS, AC section	15 checks, 0 failed	
worst magnitude	0.0674 dB of a 0.25 dB tolerance	
worst phase	2.70° of an 8.00° tolerance	

The PRBS row matters more than it looks. The model is built from a chirp — a tone sweeping upward through frequency — and then tested with a pseudo-random bit sequence it has never seen. A model that had merely memorised the input it was built from would come apart here. This one does not.

4 Case study 2 — Duty-cycle corrector

A duty-cycle corrector built around a comparator. How fast it responds is not fixed: it changes by a factor of 613,666 depending on the input level. A model with one fixed response cannot represent that at all, and the standard AC-based way of checking such a model does not work either. That combination is why blocks like this tend to have the hand-written models people trust least.

4.1 A model that follows the circuit’s dynamics

spice2rnm notices that level dependence while it is measuring the circuit, and builds a model whose speed varies with level the way the measurements say it should. That includes the circuit’s rise/fall asymmetry — it takes longer to switch one way than the other, by a measured 95 ps — which the model reproduces. Fed a clock that is high exactly half the time, the model’s output duty matches the circuit’s to 0.000 percentage points.

Transient equivalence	0.0769 against a 0.15 threshold	PASS
UVM-MS, DC section	46 checks, 0 failed	
UVM-MS, duty section	14 checks, 0 failed	
worst duty error	0.313 pp of a 0.500 pp tolerance	

4.2 Verified by function, not by formula

The testbench generated for this block has *no AC section*, and that is a measured decision rather than an oversight. An AC sweep assumes the circuit sits at one operating point and wiggles gently around it. A comparator in normal use never does that — it slams between two states. Check this model that way and it fails every single point by more than 55 dB, despite being demonstrably accurate. The failure tells you nothing about the model and everything about using the wrong check.

So the environment checks what a duty-cycle corrector is actually for. The tool drives the real circuit with clocks at several rates and several input duties, records what duty comes back out of SPICE each time, and holds the model to those recorded answers. The table spans 20 percentage points of measured behaviour — wide enough that a model ignoring its input could not fake its way through.

5 Case study 3 — Phase interpolator

A phase interpolator does not change a signal’s size. It shifts the signal in *time*: each control code slides the clock edge a little further along. So there is no frequency response worth fitting, and comparing voltages just measures noise. Conventional flows have nothing to offer this block.

spice2rnm recognises what the block does and changes approach. It measures how far the edge moves at each code, builds a delay-line model, and generates a *phase* testbench — one measured answer per control code, and no DC or AC section, because neither of those describes where an edge lands.

Codes measured	9 of 9
Total traversal	−500.1 ps, monotonic
Mean step (1 LSB)	−62.51 ps
Worst DNL	−100.5 ps (−1.61 LSB)
UVM-MS phase section	9 checks, 0 failed
worst	3.26 ps of a 6.25 ps tolerance

Two terms from that table: one LSB is one step of the control code, and DNL is how much the steps vary in size from one another. The tolerance here is a tenth of an LSB, the finest the model can resolve — tight enough to catch a real error, loose enough never to trip on rounding. The DNL figure is the circuit’s own imperfection, reported as measured. The model’s job is to reproduce it, and it does.

6 Case study 4 — Phase-locked loop

The first three are single blocks. This one is a *loop*, and that changes what a model has to get right. In a loop, a small error does not stay small — it is fed back around and pushes the whole system to a different operating point. So a model that looks perfectly reasonable on its own can still settle the loop somewhere the real circuit never goes.

It is also three different characterisation problems in one netlist, and no single method covers them. The pipeline classifies each block from its topology and routes it accordingly:

Block	Why the default does not fit	Route taken
ro_vco	oscillates with its inputs at DC — its output is a frequency, not a gain	tuning curve, scored on rate
cpump	a complementary pair driven from independently controlled gates	one $I(V)$ table per control state
lpfilt	only R and C, so its state space is exact and needs no fitting	exact state space, read from topology

Not one of those three is a simple input-to-output response. A conventional flow that fitted one to each would be fitting the wrong quantity three times over.

6.1 Checked as a system, against the transistors

A score for each block on its own is evidence about that block on its own. A loop asks a different question: wired together, do these models behave like the circuit? The co-simulation answers it directly. The design’s own testbench is run twice: once where the analog half is the generated models, once where it is the real transistors through ngspice. Only the small piece of code that connects the two halves is swapped between runs. The thresholds, the expressions and the pass/fail logic are byte-for-byte identical in both, so the two logs can be compared line against line.

Blocks modelled, three routes	3 composed, 2 more emitted
Per-block equivalence	5 of 5 pass
Supply condition	20 mV rms, declared in the deck
System check vs transistors	4 boundary signals, 0 failed
vout (control voltage)	1.86 mV, bar 0.190
aout (VCO output)	0.4% of crossings, bar 2%
dra (internal probe)	8.96 mV
Both loops lock at	400.00 MHz
settled control band	1.8468–1.8546 V vs 1.8468–1.8499 V
Model half vs transistor half	6.37 s vs 111.1 s
same testbench, same span	17.4×

The control voltage is the number to read. It is where the loop stores its state, so a model that settles anywhere else disagrees here first — and the two agree to 1.86 mV while reaching the same frequency, on a rail carrying 20 mV rms of noise the deck declares. That is looser than the figure this table carried while the rail was quiet, and the rail is the reason: a loop concentrates a supply disturbance onto precisely this node. Running the case that way is the point of it.

That figure is one draw, and the bar beside it is not. Both halves generate their own noise from the declared source, so a worst-window difference is a sample rather than a property of the model: five consecutive runs of this same deck gave 3.61, 1.83, 4.15, 2.05 and 1.86 mV here. What does not move is what the difference is judged against — 0.190 of vout’s own scale, of which 0.140 is the allowance for exactly this sampling — and that is the number the verdict is against and the record carries. An agreement figure quoted without its bar tells you less than it seems to, in the same way as a jitter ratio quoted without its correlation time.

Each node also records which of its two metrics that bar governs. A node the digital side consumes through a threshold is judged on its crossings, not on its mean. Printing the mean beside the crossing bar would read as a pass against a bar it exceeded: 0.030 against 0.020, where the number actually compared was 0.004.

6.2 The rail the deck declares is the rail the model sees

The netlist for this case says

```
vdd dd 0 dc {vcc} trnoise(0.02 1e-10 0 0)
```

— a supply with 20 mV rms on it, redrawn every 100 ps. A composition that reads that line and emits `assign dd_v = 3.3;` has deleted a disturbance the design asked for, and every check downstream then agrees about a circuit nobody is shipping. A transient of this deck puts that source at 10.91 ps rms of period jitter at the oscillator, $\pm 5\%$ at the two hundred cycles it was measured over. That figure is itself a draw. A constant rail produces none of it.

That figure is measured per design rather than quoted. Getting it needs one step first: the declared condition is restored onto the block's own supply. Extraction rebuilds every rail as a plain DC source, so the fragment a block is characterised from does not carry the noise its design states. Without that step the measurement would be of a quieter circuit than the one being modelled. The generated file says which source the number came from.

So the declared source is carried into the composed model, sampled and held at the source's own interval. That reproduces the two properties which decide how much jitter a source causes — its rms and its correlation time. The spectra agree at low frequency and part near $1/NT$, where ngspice interpolates between its noise breakpoints and the model does not.

All seven fields of a `trnoise` declaration are read, not the white pair alone. Where a deck declares flicker, the composition carries it as a Voss–McCartney summation across the octaves the run spans; where it declares a random telegraph, as a two-state Markov process with exponentially distributed dwells, at the capture and emission times given. The Voss construction produces $\alpha = 1$, and the generated file records both that and the exponent the deck asked for rather than quietly substituting one for the other. This PLL declares the white pair alone, which is why every figure here is white — $1/f$ is what dominates close-in phase noise, and a deck that states it gets it.

Carrying it changes what a comparison means. Both halves now draw their own noise, so two runs of the same circuit differ by sampling error alone and a fixed tolerance would be measuring run length. Each node's bar is widened by three standard errors of the difference between two window means, taken from the reference's own spread rather than from the declared amplitude — what actually reaches a node is a property of the circuit. On `vout` that allowance is 0.140 of the 0.190 bar, and the record carries both, so a reader can see how much of the bar was earned and how much was allowed.

6.3 Where the block works is measured, not assumed

The two output-buffer inverters sit in the clock path. Their job is to place edges in time, and an average voltage error is close to blind to that. Both pass an average-error comparison comfortably — while their error at the single worst instant is 99.9% of the full voltage swing. Those are not in conflict: if the model's edge lands slightly late, then for that brief moment the model sits at one supply rail while the real circuit sits at the other. Averaged over time it vanishes. For a clock buffer, that moment is the behaviour being verified. The pipeline classifies them from the measured DC curve — 62% and 61% of swept points within 5% of a rail is a switching characteristic, not a smooth response — and scores them on duty cycle instead.

The frequency it scores them at is measured. Where a block operates is a property of the closed loop and exists only once the digital side is attached, exactly like the operating range the golden probe already measures. The probe therefore measures each net's switching rate beside its voltage band: the two inverter inputs return 399.8 and 399.7 MHz, the 400 MHz this PLL locks to. Scored there, they pass at 0.276 and 0.307 pp of duty against a 0.5 pp bar.

The form they are re-emitted in has no delay line, so it cannot carry the measured rise/fall skew — 0.589 and 0.672 pp of duty at that clock. Both blocks measure below their own floor, so

the skew partly cancels rather than adding. That makes the floor an upper bound on what the form costs, not margin to spend, and the pipeline states it before scoring rather than leaving a pass to be read as unqualified.

That number is not one a language model can supply. Asked five times about the same inverter with the same measurements, the classifier proposed clock rates of 2 MHz, 765 MHz, 300 MHz, 10 GHz and 15 GHz; asked about the second stage, it proposed 10 GHz. The lowest is two hundred times below where the buffer runs and the highest thirty-seven times above what its own model can follow. The measurement returns the same answer every run, and gives the two stages answers agreeing to 0.03%, which no classifier run did. Intent from the model, physics from the circuit, applied to a number that looked like intent and was not.

What the classifier is asked for, it has been steady on: buffer, clock stimulus, duty metric, on every run of both stages. That is the judgement gated on a measurement it can check itself against. What moved is the one number it was in no position to know.

The verdict names the metric it came from. Asking for the classifier is not the same as getting it. Every duty figure above comes from a run with a key present and the classifier consulted. The same demonstration was then run deliberately *without* one, to see what a reader would be shown if it declined. The chirp is kept, and these same two blocks pass an rms comparison at 0.025 against a 0.15 bar — a green result from a measurement that cannot see the property in question. The two transcripts are otherwise alike, and the command line is identical, so the run records which figure each verdict was reached by, and the demonstration reports it from that record. Asking for a check on the command line does not show that it ran.

7 Results at a glance

Case study	Environment	Checks	Failed
Filter	DC + AC	49 + 15	0
Duty corrector	DC + duty	46 + 14	0
Phase interpolator	phase per code	9	0
Phase-locked loop	co-simulation vs transistors	5 + 4	0

The first three case studies — characterisation, model generation, both simulators, and every generated environment — run end to end in under 16 minutes on a laptop. The PLL is a further 15 minutes, most of it the ring oscillator’s tuning curve.

All four are one command, and `demo_4_pll.sh` runs this one. It differs from the others in what it needs, not in how it is run. Being the analog half of a co-simulation, it needs the other half: the RTL, the DPI bridge, and a *shared* libngspice — a different ngspice build from the binary the first three use. It checks for each of those before doing any work and names what is missing. A machine without them gets a skip saying which piece and where it comes from, rather than a failure twelve minutes into a run. spice2rnm’s outputs are simulator-agnostic: plain IEEE 1800 SystemVerilog, tied to no vendor. So that anyone can reproduce the results here, all of them were produced with open-source simulators — **ngspice** for characterisation and the reference transients, **xezim** for both the transient equivalence co-simulation and the generated UVM-MS environments. No commercial licence was used anywhere in this document. The generated artifacts themselves are published for inspection: the models, the complete environments, and the evidence files, exactly as the tool wrote them. They are at

<https://github.com/vrajeshprakhya/spice2rnm-demos>

under `showcase/`, stamped with the tool version and run summary that produced them. What the repository holds:

```

spice2rnm-demos/
  README.md          what this repository is, and is not
  .gitignore         run scratch; the showcase is curated, not
                    whatever a run left behind
  demo_1_lpf.sh      demo_2_dcc.sh  demo_3_pi.sh
  demo_4_pll.sh      run_all.sh
                    the four case studies, runnable end to end
harness/            8 measurement scripts the demos call
showcase/          the generated artifacts, byte-for-byte
  lpf2/            the baseline case, 14 files + equivalence/:
    rc_lpf2_rnm.sv      the model, plain real ports
    rc_lpf2_rnm_wreal.sv the same model, wreal ports
    rc_lpf2_ms_types_pkg.sv nettype real rc_lpf2_wire,
                        resolved by summing its drivers
    rc_lpf2_dms.sv      net <-> variable, interconnect ports
    rc_lpf2_bridge*.sv  MS bridge and its driving core
    rc_lpf2_proxy_pkg.sv agent <-> bridge API
    rc_lpf2_ms_pkg.sv   agent, and the SPICE goldens
    rc_lpf2_ms_scoreboard.svh the checks and tolerances
    rc_lpf2_ms_tb/test.svh environment and test
    top_rc_lpf2_ms.sv   nets, DUT, bridge, test
    run_rc_lpf2_ms.sh   compiles and runs it
    result.json         fit, verdict, warnings
    equivalence/       what the check actually ran and found
  dcc2/            same shape, 15 files + equivalence/: no AC
                    section, adds duty goldens in dcc_ms_pkg.sv
                    and the run log pipeline.txt
  pi/              same shape, 14 files + equivalence/: an MS
                    bridge, proxy and agent like the others, with
                    phase-per-code goldens and no DC or AC section
  pll/            50 files + equivalence/: five block models, three
                    interfaces, the composed analog top, the
                    node package, result.json, and env/ --
                    three per-block UVM-MS environments plus
                    the SYSTEM one, where the design's own RTL
                    and every block's model elaborate together,
                    carrying a period-jitter monitor at 1 fs
  house_lib/       4 files, NOT generated: ng_ams_pkg.sv
                    declaring nettype real ng_anet, and two
                    hand-written models using it
  lpf2_house/      13 files: lpf2 again, joined to house_lib --
                    no types package, and rc_lpf2_rnm_ng_anet.sv
                    in place of the wreal boundary
  STAMP            date, tool version, and run summary that
                    produced the artifacts above
  refresh.sh       regenerates everything; the only way this
                    directory is ever updated
  whitepaper/      this document, source and PDF

```

7.1 What the equivalence folders hold

Every case study publishes an `equivalence/` directory: what the check ran, what it found, and a manifest of what was left out.

Case	Judged on	Published verdict
Filter	rms voltage	0.0290 against 0.15
Duty corrector	rms voltage	0.0769 against 0.15
Phase interpolator	phase per code	worst 1.832 ps at code 2, tolerance 6.251 ps
Phase-locked loop	five routes, then the whole system	five block verdicts, and the system over 116,018 samples

Each folder carries three things. `verdict.txt` is lifted from that run’s own `result.json` rather than retyped, so a number here cannot disagree with the run that produced it. The simulator and SPICE logs say what actually executed. And the comparison table gives one row per measured point. The phase interpolator’s `phase_compare.csv` is nine rows, one per control code, with the measured delay, the model’s delay and the error. The PLL’s `rc_compare.csv` is the loop filter’s response to an injected current, sample by sample.

What is manifested rather than published, and why. The waveform routes write one row per simulator timestep rather than one per measured point, so the filter’s aligned comparison is 18.6 MB and its generated testbench 18 MB; the duty corrector’s waveform dump is 11.7 MB. Around 80 MB across four cases, in a repository whose history is permanent, for files `refresh.sh` regenerates in half an hour. Every one is named in `MANIFEST.txt` with its exact size and the command that recreates it, so a reader sees what exists and what it weighs without every future clone carrying it. What is published comes to 232 kB.

The distinction is drawn by size rather than by filename. A comparison table small enough to read is evidence; one large enough to be a waveform is a trace. The routes differ in which they produce — a network compared point by point writes a table, a waveform compared sample by sample writes a trace — so the rule has to be about the file, not about what it is called. And what it deliberately does not hold: the `spice2rnm` tool itself (the product is licensed separately; this repository is its evidence), and the build products of the runs — characterisation scratch, compiled simulator objects, DPI shims. `refresh.sh` filters those out, so the showcase is what a user would keep, not everything a run produces.

8 What the equivalence check compares

Every model in this document was checked the same way. The generated SystemVerilog is loaded into a digital simulator and fed an input signal. The transistor circuit is fed the very same input in SPICE. The two outputs are then lined up in time and compared point by point, and both traces are saved, so anyone can go back to the run’s own folder and work out the number again for themselves.

Two things about that arrangement matter more than the numbers it produces. The first is that **it reads the file you are actually handed**. There is a weaker number some tools report — how well the fitted maths matched its own measurements. That number never opens the generated file, and would look identical if the file had been written out wrongly. This check loads that file into a simulator and runs it, so a code-generation bug fails it.

The second is that **each block is scored on its own job**. One error figure used everywhere would be blind on most of these blocks: an average voltage error cannot see where an edge lands, and a frequency has no voltage error to measure in the first place.

If the block	then compare	and score it as
passes a signal through	the output waveform, on a sweep and on a second, unrelated input	the error, against how far the output itself moves
drives a clock	where the edges land, at the rate measured in use	duty error, in percentage points
moves an edge per setting	the crossing time at every setting	picoseconds, plus step size and evenness
oscillates	the output frequency, as the control voltage varies	a percentage of that frequency
switches a current	the current it drives, in each switch position	a percentage, for each position
is passive R/C	the output's response to a current pushed into it	the error, against how far the output moves

Every verdict records which of these rows it came from. Months later, a reader can still tell which number actually decided the result from the numbers that were merely noted alongside it.

8.1 What keeps a figure from flattering itself

Some measurements are held back. Where a block is characterised by measuring it at many points, a share of those points is kept out of the model-building and used only for scoring. The headline figure is then not the fit marking its own work. The charge pump in case study 4 reports 0.549% held back; the ring oscillator reports its held-back frequency error directly.

An input the model has never seen. The filter in case study 1 is built from a frequency sweep and then tested with a pseudo-random bit sequence it was never shown (0.0097). A model that had merely memorised what it was built from is exposed there.

The report says what range the answer covers. A comparison proves nothing about conditions it never drove, so the report states the range it did drive. The ring oscillator is the clearest case, and the honest answer there is two numbers. Inside the voltage band the loop actually operates in, its held-back frequency error is 0.002%. At the far edge of the wider range that was characterised — where the curve bends hardest, and where the loop never goes — it reaches 4.99%. Both are reported. The verdict is taken in the band that matters, and the less flattering number is printed rather than quietly dropped.

What the model shape cannot do is priced in advance. Sometimes the shape of model chosen simply cannot represent something the circuit does. When that happens, the cost is stated up front, in the same units as the verdict — as with the inverter timing asymmetry in case study 4, quoted as duty error at the clock rate in use. A pass is then read against that floor, instead of being taken as a clean win.

8.2 Blocks, and then the system

The table above scores blocks one at a time. Putting blocks together asks a further question, because errors that are small individually need not stay small once everything is wired up. Two levels of check answer it.

The first needs nothing but the netlist. The assembled model is driven at the outside edge of the design, and its output is compared against the full transistor circuit driven the same way.

The second applies when what you hand the tool is a co-simulation — a design whose analog half already talks to a digital half — and it is the one case study 4 reports. Here the design's

own testbench is run twice. Once, the analog half is the generated models. Once, it is the real transistors. Everything else is left alone: the digital logic, the limits it checks against, and the pass/fail rules are byte-for-byte identical in both runs, and only the small piece of code joining the two halves is swapped. That makes the two runs directly comparable, signal by signal: 4 signals crossing between the halves, 0 failed, the control voltage agreeing to 4.15 mV on a rail carrying 20 mV rms of declared noise, and both versions locking to the same frequency.

And a check that does not apply to a given run is reported as not applicable, with the reason why. It is never quietly counted as a pass.

8.3 One thing a score at an operating point cannot see

Every check described so far is *static*: it compares a value at an operating point. That is the right shape for most of what a model claims, and it is blind to one error that matters.

An oscillator model has to decide when to place its next edge. One way reads the control voltage at the instant the previous edge fired and schedules a whole half cycle from it. Another integrates the control across the cycle, which is what the circuit does. Both models carry the *same* tuning curve, so a check that measures frequency at held control voltages scores them identically — and it does: run against both, it passes both. Give them the same moving control and they part: on the reference ring, fed a control with a 100 ps correlation time, the sampling form produced $3.37\times$ the period jitter the transistors did.

How far apart they land is not a fixed property of the two forms. It is $\sqrt{T_{\text{half}}/\tau}$ in the correlation time τ of the disturbance, which for 1.25 ns against 100 ps predicts 3.54 — the measurement above, to within 5%. A faster disturbance separates them further, so a ratio quoted without its τ says less than it appears to.

So the tool checks the dynamic behaviour directly. A known disturbance is injected in SPICE at several amplitudes, and the disturbance *ngspice actually solved* is replayed into the model sample for sample. That last part is the point: giving the model its own random noise of the same size would compare two random sequences, and agreement between those says more about how long the run was than about the model. Replaying the same waveform compares transfer.

Both of the model's inputs are exercised, not just the obvious one. On the reference ring the supply moves the frequency by 450.6 MHz/V, comparable to the control port and opposite in sign, so a model given no supply input cannot show what a droop does at all. The verdict is the worse of the two paths.

Path	Injected	Circuit	Model	Ratio
either	none (floor)	0.051 ps	0.000 ps	not judged
control	5 mV rms	2.396 ps	2.352 ps	0.981
	20 mV rms	8.743 ps	8.690 ps	0.994
	50 mV rms	26.719 ps	25.618 ps	0.959
supply	5 mV rms	2.808 ps	2.640 ps	0.940
	20 mV rms	11.617 ps	12.139 ps	1.045
	50 mV rms	28.422 ps	27.656 ps	0.973

These ratios are a draw, like the boundary figures earlier. Each injection is a fresh realisation and an rms over two hundred cycles carries about 5% on each side, so the same six points on the previous run read 0.987, 0.946, 0.925 and 0.896, 0.953, 0.970. That spread is why the bar is 0.25 rather than something that looks stricter: a tolerance tighter than the measurement's own uncertainty fails correct models at a rate set by run length.

The floor row is measured and deliberately not judged. With nothing injected the reference still reports about 54 fs, which is how precisely the edge-finder can locate a crossing rather than anything the circuit did. Asking the model to reproduce it would be asking it to reproduce

an artefact. It is used instead as the floor every other amplitude has to clear before its result counts.

That agreement was not obtained by fitting anything. Take the supply path, where the sensitivity is separately measured: 450.6 MHz/V on a 2.509 ns period predicts 14.18 ps of period jitter for 5 mV rms, if the disturbance held still across the cycle. The circuit gives 2.797 ps — a ratio of 0.197, against the $\sqrt{\tau/T} = \sqrt{100.3 \text{ ps}/2509 \text{ ps}} = 0.200$ that averaging over the cycle predicts.

It holds across the decade rather than at one point: 0.197, 0.223 and 0.203 at the three judged amplitudes, where an oscillator that did not average its input would sit at 1 — five times higher. Three points within 12% of a parameter-free prediction, over a tenfold range, and each carrying the same few per cent of sampling uncertainty as the ratios above. The model reproduces it because it integrates, and the version that samples does not.

And it decides for itself whether the check is worth running, because for most designs the honest answer is no and a quarter of an hour is a real cost. Two things can say yes.

The first is the design's own closed-loop behaviour, measured: how far the control moves within one oscillator period, carried through the tuning curve's own slope into period movement, against what the reference measurement can resolve. On this PLL it says no, and says it with numbers — on a run where it was the deciding voice, 12.18 μV rms per 2.503 ns period, worth 0.0288 ps against a 0.0521 ps floor, 0.55 $\times$ what the reference can resolve. A loop filter holds its control still; that is what a loop filter is for.

The second is the netlist, and it is what runs the sweep above: this deck declares 20 mV rms on the ring's supply, and a design that writes noise into its own deck is asking for noise to be simulated. That rule is matched on the NET rather than the source name. Block extraction rebuilds every supply as a plain DC source: the fragment carries `VBIAS2 dd 0 DC 3.3` where the top level wrote `vdd dd 0 dc {vcc} trnoise(...)`. The net survives and the source name does not, so matched on the name the rule was unreachable on every hierarchical run: on every run, that is, which has a top-level deck to declare anything in.

There is no flag to force it, because a request to run a check is not evidence that one is warranted. And where the tool has no closed-loop evidence either way, it runs the check rather than skipping it: having no opinion about whether a check is needed is not the same as deciding it is not.

What the sweep measures, and what it does not. The ring in isolation, driven on its own characterisation fragment. That is the right scope for the question it answers — does this model transfer a disturbance correctly — and it is not the same question as what the loop does with one. Inside the loop the filter holds the control still, which is precisely why the closed-loop half of the gate declines; these numbers come from driving the block directly, where nothing is holding it. A reader should not read the supply column as a statement about this PLL's supply rejection. It is a statement about the model.

9 Why a green testbench is worth believing

Generated verification has an obvious credibility problem. If the same tool writes the model *and* writes the test, what stops it from writing a test its own model is bound to pass? Three rules answer that.

The right answers come from SPICE, never from the model. Every expected value in every generated environment is a SPICE measurement of your circuit. Checking a model against the equations that model itself implements would pass no matter how wrong the model was, so spice2rnm never does it.

Every environment is proven able to fail. The tool deliberately breaks the generated model and confirms the test goes red. It also *measures* how small a break it can still catch,

rather than simply claiming one. For the duty-cycle corrector’s environment:

Injected fault	Detected by
comparator offset, 20 mV	DC and duty sections
dynamics slowed 10×	duty section (DC is structurally blind)
dynamics slowed 50×	duty section (DC is structurally blind)

The right-hand column is the point. A DC check looks at the circuit after it has settled, and that is the whole of what many hand-written environments test. It cannot see *any* error in how fast a block responds — slow this model down fiftyfold and the DC check still passes it. The function-matched sections exist because of that blind spot, and the fault sweep is re-run against the shipped model so these detection claims stay true rather than becoming folklore.

The test is checked in both directions. A test that fails everything is as useless as one that passes everything. So the same sweep also confirms that errors small enough to be acceptable do *not* trip the environment.

10 Functional specifications as input

A netlist says what a circuit *is*. It never says what it is *for*, what rate it must run at, or how close is close enough. Hand those to the tool as a plain-language document — a datasheet excerpt, a requirements page — and it reads out the facts that shape verification:

From the specification	Used for
operating clock range	where the environment drives the block
accepted input duty range	the stimulus sweep
duty accuracy requirement	the pass/fail tolerance
block function	which environment is generated

On the duty-cycle corrector, the specification-driven run checks 35–65% input duty at the specified clock rates against the specification’s own 1.0 pp requirement: 15 checks, 0 failed, worst 0.428 pp. **PASS**

The specification is cross-checked, never trusted. Every extracted fact is validated against what the circuit was actually measured to do, and the measurement wins. When the two disagree, that disagreement is surfaced as a finding:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

That is a coverage gap between a requirement and a characterisation, caught at model-generation time instead of at silicon correlation — and it is the kind of finding no amount of manual model review produces, because no one document contains both halves of it.

11 Joining an existing model library

A team with real-number models already has conventions — which user-defined nettype analog values travel on, which package declares it, what discipline their model ports use. Generated files that import their own freshly-invented nettype do not join that library; they sit beside it, each net scheme blind to the other, and the integration cost lands on the customer.

Point `--style-from` at the library and the output conforms: the environment’s analog nets use *their* nettype, its files import *their* package, no types package of the tool’s own is emitted, and the generated run script compiles their package file. A library whose models use `wreal` ports

gets the `wreal` wrapper without being asked. Their timescale is reported but deliberately not adopted — generated delays are computed femtosecond values, and timescale is per compilation unit, so the directives interoperate unchanged.

Reading a house style is a judgement, so it is read and then verified. A library may declare several nettypes, only one of which is the living standard — and what says so is often a comment. Measured on a library that declares two, where the deprecated one happens to sort first alphabetically:

mechanical scan	picks the deprecated nettype, and warns
model reading	picks the live one, quoting the deprecation notice
verification	re-checks the choice against the named file

The model’s stated grounds, verbatim from that run: *“line 15 marks `aaa_legacy_wire` deprecated (“kept only so ancient testbenches still compile. Do NOT use in new code”), ... both models declare all analog ports as `acme_ewire`, confirming this is the live convention.”* Nothing in that reading is taken on trust: the named nettype, package and file are checked against the file before use, and a claim the file does not support leaves the mechanical answer standing, out loud. Where the tool has a *measurement* — a count of which discipline the library’s ports actually use — the count wins outright and a disagreement is recorded rather than adopted.

The result is an environment that runs on the house net, and it is published rather than described. The showcase carries the same filter twice: `lpf2/` generated normally, and `lpf2_house/` generated against `house_lib/` — a small hand-written library standing in for a customer’s own. Diffing the two directories is the whole argument:

	<code>lpf2/</code>	<code>lpf2_house/</code>
types package	emitted	not emitted
nets	<code>rc_lpf2_wire</code>	<code>ng_anet</code>
imports	the generated one	<code>ng_ams_pkg</code>
run script	compiles it	compiles the house package
model ports	plain <code>real</code>	<code>ng_anet</code> , and the environment instantiates it
the model	byte-identical	
the scoreboard	byte-identical	

Conformance changes the boundary and nothing else — and the published environment runs from where it sits, against the library beside it: 49 DC and 15 AC checks, 0 failed, exactly the default environment’s result.

Note what the environment instantiates in that second column. When the model is delivered behind a port boundary, the generated testbench drives *that* module rather than the core inside it, and the boundary’s file joins the compile list — so what the evidence covers is what ships, not a component of it. Measured separately, a library resolving its net by averaging rather than summing conforms and passes the same way, and a library using `wreal` ports gets the `wreal` wrapper instead. Without `--style-from`, or against a directory with nothing recognisable in it, nothing changes — and the run says so.

12 AI where it helps, physics where it decides

spice2rnm asks a language model four kinds of question: where one block ends and the next begins, what a block is for, what a written specification requires, and what conventions an existing model library follows (Section 11). Every one of those is a question of *intent*. None is a question of physics.

That is the rule the tool works by: **intent from the model, physics from the measurement**. Every suggestion is checked against the measured circuit before it is used. One that does not check out is discarded, the run carries on with what was measured, and it says that it did so. With no API key at all, every AI feature declines cleanly and the pipeline runs entirely on measurement.

Which of those two happened is recorded in the run’s own result, not only in its log. A verdict carries the metric it was reached by. A report read months later can then tell a check that ran from one that declined and fell back. Without that, every green result would have to be taken on trust that the feature requested was the feature exercised.

13 Scope and requirements

- **Inputs:** SPICE netlists, flat or with subcircuits; optional plain-language functional specification; optionally an existing RNM library to take conventions from.
- **Outputs:** SystemVerilog real-number models, with the port discipline of your choice — plain `real` (portable anywhere), `wreal`, or your library’s own nettype; UVM-MS 1.0 environments; equivalence reports with archived waveforms.
- **Toolchain:** a SPICE simulator for characterisation, and a digital simulator capable of running real number models — with UVM support for the generated environments. The generated models and environments are standard SystemVerilog and carry no vendor dependence, so either may be the one your flow already uses.
- **Block-level scope:** single-input single-output blocks and compositions of them, including current-mode blocks sharing a node. Hierarchical composition trades some accuracy for structure; the equivalence report states the composed figure separately from the per-block figures, so the trade is visible, not hidden.
- **Jitter TRANSFER is verified; intrinsic device noise is characterised.** Both are measured, and they are measured differently.

What is checked is what the model does with a disturbance it is given: injected in SPICE, replayed into the model, period jitter compared, on both the control and the supply. Section 8.3 gives the measurements — six judged points over a tenfold range, worst ratio 0.940 against a 0.25 tolerance, reached by the pipeline’s own decision because the deck declares noise on a net the ring is wired to. This is a real check with a verdict, and it fails a model whose transfer is wrong.

The oscillator’s own noise is characterised through its impulse sensitivity function, which the pipeline measures without being asked: a narrow charge injected at each phase of the cycle, the lasting phase shift measured, and $\Gamma(\phi)$ is the sensitivity that results.¹ On the reference ring Γ_{rms} is 4.04×10^{12} rad/C from the automated sweep, against 4.33×10^{12} measured by hand: the two agree to 7%. Γ changes sign across the cycle as a ring’s must, and halves when the injected charge halves.

Γ_{rms} times a device noise density is a jitter: 0.236 to 0.578 ps rms here, or -114.5 to -106.7 dBc/Hz at 1 MHz, reported as a range because the channel-noise factor and the way

¹The method is Hajimiri, Limotyrakis and Lee, *Jitter and Phase Noise in Ring Oscillators*, IEEE JSSC 34(6), June 1999. Two details decide whether it works, and each returns a plausible wrong answer rather than an error, so the tool enforces both. The pulse is applied differentially where the circuit is differential — into one node of a differential ring it is mostly common mode, which the ring rejects by design, and the measurement then reads its own floor however large the pulse. And linearity is judged at the phase of greatest sensitivity rather than a convenient one: the same charge that moves the phase by 0.04 ps where the ring barely responds moves it by 52 ps at the peak.

per-node densities combine are both ranges. The model carries it as a parameter, shipped at zero: the figure is computed from the measured ISF rather than compared against a reference transient, since ngspice simulates no device noise in one.

Driven blocks jitter differently, and the models say so. An oscillator measures each edge from its own last one, so its phase random-walks and the error accumulates. A buffer or a lag block measures its output from an *input* event, so every traversal draws independently and nothing accumulates.² The scheduled-lag models take a synchronous-jitter figure on that basis, perturbing the input-to-output delay once per traversal. That figure is supplied, not measured: a transient of a deck declaring no noise has no delay variation in it to measure. The generated file says so. It also states the constant latency the perturbation costs, because a tap only looks backwards, so an edge can be early only relative to a delay already present.

Edge JITTER is measured where the reference has edges. On the reference PLL’s two clock-path buffers it is 3.98 ps and 3.60 ps, reported beside the skew because the two fail differently. It is reported rather than judged: there is no threshold for it, and inventing one from a single design would be worse than showing the number. The system equivalence check does not judge it either, and the reason is the probe rather than the model. Its boundary log is sampled at 100 ps. The composed model’s output crosses its threshold between samples, so a crossing can only be located to the nearest tick. Reporting that as jitter would be measuring the sampling grid.

The generated UVM-MS environment carries a monitor without that limitation. It is event-driven on the signal rather than sampled on a clock, and compiled at 1fs while the rest of the environment stays at 1ps, which costs nothing anywhere else because SystemVerilog scopes timescale per module. Running that environment on the composed PLL gives 12.70 ps rms of period jitter over 1151 periods at the boundary, with none of its system checks failed and one inconclusive, over 116,012 boundary crossings. The number of checks is not quoted because it is not fixed: one of them exists only where the transistor reference settled measurably, which under a noisy rail it may or may not do.

REPORTED, NOT JUDGED, and for a specific reason rather than caution. A loop suppresses its oscillator’s jitter inside the loop bandwidth, so a figure measured at the boundary is not the block’s own jitter and the two are not expected to agree. The 10.91 ps this deck’s declared rail is worth at `ro_vco` is a different measurement of a different quantity. On this run they differ by 16%; on the one before, by 3%. Both are draws, and neither is evidence about the other.

This is further than published practice goes on the validation side, while the gap named above — intrinsic noise in the model — is one the same practice also has. Analog Devices’ production RNM suite for a GHz transceiver validates its PLL on lock time and frequency error, the same quantities checked in Section 8. It states that “model validation remains an area for improvement in the real number modeling flow”, and names frequency-domain performance as future work.³

²The distinction and both implementations are Kundert’s, in *Modeling and Simulation of Jitter in Phase-Locked Loops*, AACD 1997. He makes four points this model follows. Driven systems jitter without accumulating; autonomous ones have a phase difference that is “a random walk that is not bounded”. The metric for both is $J = \sigma(T)$, the standard deviation of one period. Driven jitter is modelled by “randomly dithering the delay argument ... on every output transition”. And the per-update dither scales by $\sqrt{N}$ for N updates per period, which is 1.414 for the two this model uses.

Each was checked. Asked for 1% rms period jitter, the model produced 0.9998%. Its scaling constant is the $\sqrt{2}$ his listing specifies. And over 30,000 cycles, the variance of the accumulated deviation grew linearly in cycle count, at 1.004 ± 0.026 of the kJ^2 his random walk requires.

The check also found a defect. At one sub-step per half cycle the scheduler cannot resolve a randomised threshold. The model now refuses that combination, rather than reporting 39.6% jitter where 1% was asked.

³J. Dudek, *Real Number Modeling of RF Circuits*, DVCon. The paper notes that because RNMs exist to enable system-level testing, “the requirements for model validation are greatly relaxed in this environment”.

© 2026 Veylon Systems. spice2rnm is a Veylon Systems product. Every figure in this document was produced by the shipped tool from its bundled demonstration circuits, most recently on 2026-09-16. The demonstration suite at <https://github.com/vrajeshprakhya/spice2rnm-demos> regenerates them in one command.

Veylon Systems